Reproducing COCONUT and Testing Written Steps Between Latent Thoughts

Eran Ben Eliyahou (eran.b@campus.technion.ac.il)[1]
[1]Technion – Israel Institute of Technology

Abstract

When people solve a problem, they rarely do all the thinking first. They think, write down a partial result, and go on from there. We ask whether a COCONUT model can work this way. On GSM8K we reproduce a released GPT-2 COCONUT model (27.1% with six thoughts; paper: 34.1%) and SIM-CoT, whose thoughts can be read as text (44.7%; paper: 44.8%). We then let SIM-CoT write down its first step and think again. After a short fine-tune on its own steps, accuracy with its own written step rose from 34.4% to 41.5%, and with a correct step from 38.6% to 45.4%, the level of silent thinking alone. The same training without steps changed little. This suggests that thinking in turns can be learned with modest training.

Keywords: latent reasoning; chain of thought; continuous thought; interleaved thinking; reproduction

Introduction

When people work through a math problem, the thinking is spread over the whole process. We think for a moment, write down a partial result, and go on with the note in front of us. Writing things down takes load off working memory, and people do it more as tasks get longer. Even silent thinking is partly verbal, with inner speech mixed into less verbal thought.

Language models have their own version of the written note. With chain-of-thought prompting, a model writes out its steps before the answer and becomes more accurate on multi-step problems (J. Wei et al., 2022). COCONUT (Hao et al., 2024) goes the other way. In *latent mode* the model feeds its last hidden state back as the next input embedding and writes no words. These vectors are called *continuous thoughts*. On the GSM8K math word problems (Cobbe et al., 2021) with GPT-2, the paper reports 34.1% accuracy, compared with 16.5% when the model answers directly.

This project has two parts. In **Part 1** we reproduce COCONUT's main result. Training COCONUT was beyond our compute, so we used an open-source GPT-2 COCONUT model (tsrigo, n.d.). In **Part 2** we build a small proof of concept of mixed thinking. The model thinks, writes down its first step, and then thinks again with that step in its input.

The COCONUT trend reproduced, at a lower level than in the paper (27.1% vs. 34.1%), and SIM-CoT (X. Wei et al., 2026) reproduced closely (44.7% vs. 44.8%). In Part 2, the released model was not ready for written steps. It had never seen text between a question and its thoughts, and at first any step we added lowered its accuracy. A short fine-tune in which the model saw its own written steps changed this. Accuracy with its own step rose from 34.4% to 41.5%, and with a correct step from 38.6% to 45.4%, the same as silent thinking alone. A control model that got the same training without steps stayed where it was. The natural next goal is for the written step to add accuracy on top of silent thinking, and our results suggest how to get there. We also list the attempts that failed along the way, since they shaped the final design.

Background

COCONUT

COCONUT has two modes. In language mode it is a normal language model. A marker token `<bot>` switches to latent mode. There, the last-layer hidden state at each position is used as the input at the next position, and no word is produced. A marker `<eot>` switches back. Training follows the iCoT curriculum (Deng et al., 2024) on an extended version of GSM8K from the same authors, where each solution step is an equation such as `«16-3-4=9»`. At stage k, the first k written steps are replaced by $k \cdot c$ thoughts ($c = 2$ for GSM8K). The final model only ever sees

question → `<bot>` 6 thoughts `<eot>` → `### answer`.

The loss covers only the text after the thoughts, and gradients flow back through the thoughts.

SIM-CoT

SIM-CoT (X. Wei et al., 2026) adds supervision at the level of single steps. During training, a second GPT-2, the *explainer*, learns to turn each pair of thoughts back into the written step it replaced. With GPT-2 the paper reports 44.8% on GSM8K, compared with 36.6% for its own COCONUT run, using up to 10 thoughts (5 pairs). The released model still includes the explainer, so we can read the model's thoughts as text. Part 2 relies on this.

Part 1: Reproduction

Method

Models. We use a COCONUT GPT-2 model (`tsrigo/coconut`, `stage4_checkpoint_25`, which matches the paper's GSM8K schedule), and the SIM-CoT model (`internlm/SIM_COT-GPT2-Coconut`).

Data. All 1319 GSM8K test questions, in the official COCONUT input format.

Procedure. We keep the weights fixed and change only the

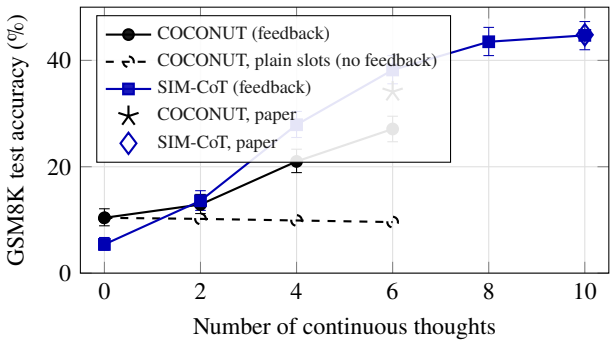


Figure 1: Part 1. GSM8K accuracy (1319 test questions) by number of continuous thoughts, with the weights fixed. Error bars: 95% Wilson intervals.

number of thoughts at inference: 0 to 6 for COCONUT, 0 to 10 for SIM-CoT. Decoding is greedy, with at most 64 new tokens. An answer is correct if the text after the last `#` matches the true answer (commas removed), as in the official evaluation script.

Code. The COCONUT code handles a single group of thoughts in the input, but we need two thinking stages, one per round. Thus, we treated each round as its own input with one block. **Control.** In the *plain* control the thought slots are still there, but each keeps the learned embedding of the `<latent>` token instead of the fed-back hidden state. Only the feedback is removed.

Results

COCONUT. Accuracy goes up with the number of thoughts, from 10.4% to 27.1% (Figure 1), which matches the paper's main trend. At six thoughts we are below the reported 34.1%. We suspect it might be due to the model being trained by another group.

Feedback control. When the hidden states are not fed back, the extra slots do not seem to help, and accuracy stays near 10% (Figure 1). With six thoughts, the feedback adds 17.5 points. So in our runs the gain seems to come from the feedback itself, and extra positions alone did not help.

SIM-CoT. With 10 thoughts SIM-CoT reaches 44.7%, similar to the reported 44.8%. Accuracy grows with each pair of thoughts and levels off between 8 and 10. With no thoughts it scores below COCONUT (5.4% vs. 10.4%), so it seems to lean on its thoughts even more. The explainer turns the thoughts into readable steps. For one test question the true steps are `«3*68=204»`, `«2*80=160»` and `«6*55=330»`. The decoded thoughts give exactly these three, followed by `«204+160+330=690»`, which is off by 4 from the true sum.

Follow-up Ideas

We considered four follow-ups.

(1) *Dynamic thinking length*: pick the number of thoughts for each question, either with a classifier (e.g. 1-9) or with a binary choice to stop or continue thinking.

Table 1: Experiment 1, answers for the first five test questions (taken in order, not picked); ✓ = correct.

Q	True	No step	Own	True	Own in.	True in.
1	18	18 ✓	22	36	54	18 ✓
2	3	3 ✓	3 ✓	3 ✓	3 ✓	3 ✓
3	70000	45000	10000	45000	45000	45000
4	540	180	540 ✓	540 ✓	360	243
5	20	20 ✓	11	20 ✓	8	20 ✓

(2) *Written steps between latent thoughts*: let the model write down a step and then think again.

(3) *Reading and steering thoughts*: decode thoughts to see what the model computes, or nudge them via steering, for example to evade refusals or to insert or remove biases.

(4) *Thought distillation*: a teacher COCONUT model distills a student base model on the thinking tokens. This can also be checked for cross-model training. We chose (2) because it is the closest to the way people seem to handle longer problems, alternating between silent thinking and writing things down.

Hypothesis. If the model can see its first step written down, the second round of thinking may start further along and have less to keep track of, so accuracy may go up.

Shared setup. Both experiments use SIM-CoT GPT-2, since it has working thoughts and an explainer that can write them out. We use all 1319 test questions, paired across settings. Each round of thinking gets the full trained budget of 10 thoughts, and the answer always comes from the model itself. There are three basic settings.

No step: the model reads the question, thinks with 10 thoughts, and answers.

Own step: the first round is the same. The explainer then turns the first pair of thoughts into a written step s_1, for example `«3+4=7»`.

In the second round the model reads the question, a new line, and s_1, thinks again with 10 thoughts, and answers. *True step* is the same as own step, except that s_1 is the true first step from the dataset. This removes the explainer's mistakes, so it shows the effect of the layout on its own. The model's own first step matches the true one word for word in 42.5% of test questions. This might undercount a bit, as `«4*20=80»` and `«20*4=80»` count as different.

Part 2

Experiment 1: Prompting Only

We first gave the steps to the released model as it is. We tried two positions. In the first, the step comes after the question (own and true step). In the second, the step is written as a plain equation (`16-3-4=9.`) and placed inside the question, just before its last sentence, so that it reads like a given fact (own and true step *inside*). For example:

> . . . She sells the remainder at the farmers' market daily for $2 per fresh duck egg. `16-3-4=9.` How much in dollars does she make every day at the farmers' market?

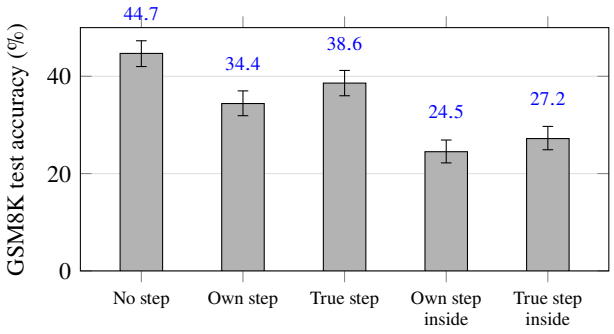


Figure 2: Experiment 1, the released SIM-CoT model before any extra training. The model was not used to written steps yet, and each way of adding one lowered accuracy. Error bars: 95% Wilson intervals; 1319 questions.

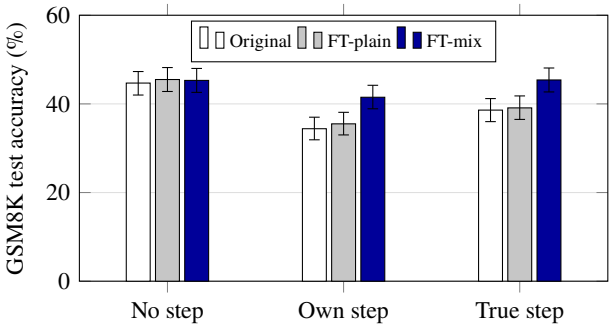


Figure 3: Experiment 2. After training with written steps (FT-mix), accuracy with a correct step reaches the level of silent thinking, and accuracy with the model's own step comes close to it. The same training without steps (FT-plain) changes little. Error bars: 95% Wilson intervals; 1319 questions.

Results. In our runs every variant scored below no step (Figure 2). With the own step, accuracy dropped by 10.2 points. With the true step it dropped by 6.1 points, so even a correct step hurt. Inside the question the drops were larger: 20.2 points for the own step, and 17.4 for the true step.

Because the true step also hurt, we suspect the main problem is the layout more than the step itself. The model had never seen text between the question and its thoughts. A small model is unlikely to adjust to a new layout from the prompt alone, so we tried training it.

Part 2

Experiment 2: Training on the Step Layout

Method. We trained SIM-CoT a little further so that it would see written steps in its input. We made two fine-tuned variants.
FT-mix saw half of its training questions (by coin flip) with its own first step written after the question, and the other half without a step, to avoid catastrophic forgetting.
FT-plain saw the same questions in the same order, always without a step. FT-plain tells us how much the extra training changes the model by itself.
Steps. The steps were the model's own, decoded by the explainer from the original model's first pair of thoughts. This way, training looks like the test, where steps are sometimes wrong. The answer target was always the true answer.
Data. We drew 50,000 questions at random from the 385,620 training questions. The test set was never used for training. On these training questions the own step was fully right in 80.0% of cases, much more often than on the test questions (42.5%), possibly because the SIM-CoT explainer was trained on them.
Training. We used the loss of COCONUT's final stage: only the answer text (`### N`) is graded, and gradients flow through the 10 thoughts. We used the official COCONUT forward pass and batching, AdamW, a learning rate of 2×10^{-5} with 100 warm-up steps and linear decay, batches of 32, and one pass over the data (1,563 updates, about 2 hours per model on one L40S, which we eventually got after starting on an RTX 2080).

Table 2: Experiment 2: accuracy (%) and the effect of the step (step minus no step, paired, 95% CI).

Model	No	Own	True	Own − No	True − No
Original	44.7	34.4	38.6	−10.2	−6.1
FT-plain	45.5	35.5	39.1	−10.0	−6.4
FT-mix	45.3	41.5	45.4	−3.9	+0.1
Effect of the step, FT-mix − FT-plain				+6.1	+6.4

Test. All three models (original, FT-plain, FT-mix) were tested in the same three settings. Every model got the *same* saved own step, so only the model changes between them.
Checks. The training inputs are identical to the test inputs. Re-running the original model gives exactly the numbers of Experiment 1 (589, 454 and 509 correct).
Results. After training, the model handled written steps much better (Figure 3, Table 2). Compared with the original model, FT-mix gained 7.1 points with its own step and 6.8 points with the true step. With the true step it now scores the same as with silent thinking alone. With its own step it comes within 3.9 points of silent thinking, where the gap was 10.2 points before.

FT-plain stayed close to the original model in every setting, so the change in FT-mix seems to come from seeing the steps during training. The effect of the step was about 6 points better in FT-mix than in FT-plain, for both the own and the true step. On the own-step setting alone, FT-mix scored 6.0 points above FT-plain. Thinking alone was not hurt by the training: no step stayed between 44.7% and 45.5% in all three models, with no clear difference between any two of them.

So after a short training, the model takes a written note in its input and keeps its accuracy. The step does not yet push accuracy above silent thinking, which is the second half of our hypothesis.

Discussion

Why the step hurt at first. Before our training, the model had seen one layout only: question, thoughts, answer. Even in the earlier curriculum stages, written steps came after the thoughts and never between the question and the thoughts. We think this explains most of the early drop, as the true step hurt too, and a short training with steps removed most of the drop while the same training without steps did not. Placing the step inside the question hurt most, perhaps because the new numbers looked like extra context to use. Language models are known to be thrown off by added context (Shi et al., 2023). Prompting is unlikely to fix this on its own, since GPT-2 has only 124M parameters. Following examples in the prompt shows up mostly in much larger models (Brown et al., 2020). In an earlier attempt with three worked examples, COCONUT GPT-2 kept writing only `### N` (row 7 of Table 3).

What could make the step pay off. We see three likely reasons why the step does not add accuracy yet.

First, the model may trust its notes too much. In training, its own step was right 80.0% of the time, but on test questions only 42.5%, which fits the small remaining gap for the own step.

Second, our test left little for the note to do. The second round still had all 10 thoughts, and the first step is often the easiest one. A future direction might be to extend this method to the next thoughts and steps too.

Third, the training was short: one pass over 50,000 questions, so there is room to train longer.

A small step toward mixed thinking. People seem to gain most from writing things down when a problem is too long to hold in mind. Our test was the case where a note should matter least: one note, and a full budget of thinking after it. Even so, one short training run was enough for the model to take its own notes on board and keep its accuracy. This was the main obstacle we found, and it turned out to be something training can change. We find this encouraging for longer chains, where the notes may matter more.

Limits. These results come from one model, one short training run, and only the first step. They show that a model can learn to take in written steps with little training. We have not yet shown that accuracy can be raised.

Future Work

Several next steps follow from the fine-tune. The steps used in training could be made as reliable as the ones the model will meet at test time, for example by decoding them with an explainer that never saw those questions, so that the model learns how far to trust a note.

Alternatively, the second round could get fewer thoughts, to see whether a written step can save silent thinking. This is how notes seem to help people, and it might improve efficiency even at equal accuracy. A fuller version would run several rounds, each adding a new step written by the trained model, with the explainer trained alongside so that its steps stay faithful.

What We Tried Along the Way

Our final design came out of a series of attempts that failed. Table 3 lists them in order. Each one changed what we did next. Attempts 1 and 6 moved us to released models and showed that we could not avoid needing a cluster GPU.

Attempts 4 and 7 taught us to test a model in its own format first.

Attempts 10 and 11 led to the training in Experiment 2. Rows 1 to 3 are documented in project notes, and the others have code, logs and results in the repository; due to clutter, I am submitting only the clean final version.

Conclusion

We reproduced COCONUT's main result at inference time. Continuous thoughts raised GPT-2's GSM8K accuracy from 10% to 27%, and in our runs the gain needed the hidden-state feedback. SIM-CoT reproduced as well, and its thoughts can be read as steps.

We then built a small version of mixed thinking, where the model thinks, writes down a step, and thinks again. The released model first had to get used to this layout. After a short training with its own steps, it read a correct step with no loss in accuracy and handled its own steps much better (34.4% to 41.5%). Making the written step add accuracy on top of silent thinking is the next step, and we see these results as a promising start for models that think and take notes in turns.

Acknowledgments

Use of AI. We used Claude and AntiGravity throughout the project. Under our direction, they wrote the experiment code and the draft of the report. They also read the logs and summarized the saved result files. I chose the paper and the follow-up ideas, and decided what to expand on. I also came up with the variants and all the plans for the experiments. I also read and manually approved every plan before it was implemented, and manually revised this report. All numbers in this report come from the result files in the repository.

Code names in code: *no step* = L, *own step* = T2, *true step* = T2-gold, *inside* = T2-in.

References

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. *Advances in neural information processing systems*, *33*, 1877–1901.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. (2021). Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Deng, Y., Choi, Y., & Shieber, S. (2024). From explicit cot to implicit cot: Learning to internalize cot step by step. *arXiv preprint arXiv:2405.14838*.

Table 3: What we tried along the way, in order. Accuracies are on GSM8K unless noted.

#	What we tried	Result	What went wrong and what we learned
1	Train COCONUT ourselves (GPT-2, ProsQA, official code, 8 GB GPU)	stopped; over 3 days estimated	The GPU was too small. We moved to released models and inference only.
2	Train GPT-2 to switch into latent mode by itself	1.44% vs. 8.72% (fine-tuned baseline)	Training was unstable and the model repeated itself. It did place the switch at plausible points, which is very promising. A new behaviour seems to need a gradual curriculum.
3	Split questions into sub-questions with a strong LLM, then ask GPT-2	about 8% (100 questions)	The small model got confused when we asked about a smaller detail within the larger question; it sometimes gave the final answer and then did not know what to do.
4	EntailmentBank proofs, one block of thoughts per proof step	97% proof structure, with or without thoughts	The structure came from our script, which supplied the true proof links. The model wrote random digits, since it was fine-tuned on GSM8K.
5	A public Qwen2.5-1.5B "COCONUT" model, two-stage step prompts	12% vs. 6% (100 questions)	Not a real test: hidden states were never fed back, and the thought token was plain text, not even in the vocabulary. We learned to check that the method is really running.
6	Larger models on the 8 GB GPU (Qwen2.5-7B, Gemma-3-4B)	did not fit; over 25 h per pass with offloading	Hardware. We finally got a strong GPU (L40S).
7	COCONUT GPT-2 with the same two-stage step prompts and 3 worked examples	0.9 to 3.4%	The model only wrote `### N`. A 124M model did not pick up a new format from three examples. We learned to test a model in its own format first.
8	COCONUT Qwen2.5-7B model	meaningless text, even with no thoughts	The upload seems broken. Short checks before a full run caught it.
9	COCONUT Qwen3-4B model	always Yes	It seems to have been trained on an undocumented yes/no task, as the model card was lacking too.
10	SIM-CoT, written step after the question (Exp. 1)	34.4% / 38.6% vs. 44.7%	The layout was new to the model, and even the true step hurt.
11	SIM-CoT, written step inside the question (Exp. 1)	24.5% / 27.2% vs. 44.7%	Changing the question hurt more. Prompting alone did not seem enough, so we trained the model (Exp. 2).

Hao, S., Sukhbaatar, S., Su, D., Li, X., Hu, Z., Weston, J., & Tian, Y. (2024). Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*.

Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E. H., Schärli, N., & Zhou, D. (2023). Large language models can be easily distracted by irrelevant context. *International conference on machine learning*, 31210–31227.

tsrigo. (n.d.). Coconut: Checkpoints of facebookresearch/coconut [Checkpoint `stage4_checkpoint_25`]. https://huggingface.co/tsrigo/coconut

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, *35*, 24824–24837.

Wei, X., Liu, X., Zang, Y., Dong, X., Cao, Y., Wang, J., Qiu, X., & Lin, D. (2026). Sim-cot: Supervised implicit chain-of-thought. *International Conference on Learning Representations*, *2026*, 56721–56742.